

OUC26夏移动软件开发-实验6

姓名：马一诺 学号：24020007088

项目	内容
姓名和学号	马一诺，24020007088
所属课程	中国海洋大学26夏《移动软件开发》
实验名称	实验6：小程序云开发——图片分享社区
博客地址	https://blog.csdn.net/2604_96828618/article/details/164729942?fromshare=blogdetail&sharetype=blogdetail&sharerId=164729942&sharerefer=PC&sharesource=2604_96828618&sharefrom=from_link
代码仓库地址	https://github.com/1-qaz-2-wsx/ouc26-mobile-dev/tree/main/lab6/code6

一、实验内容

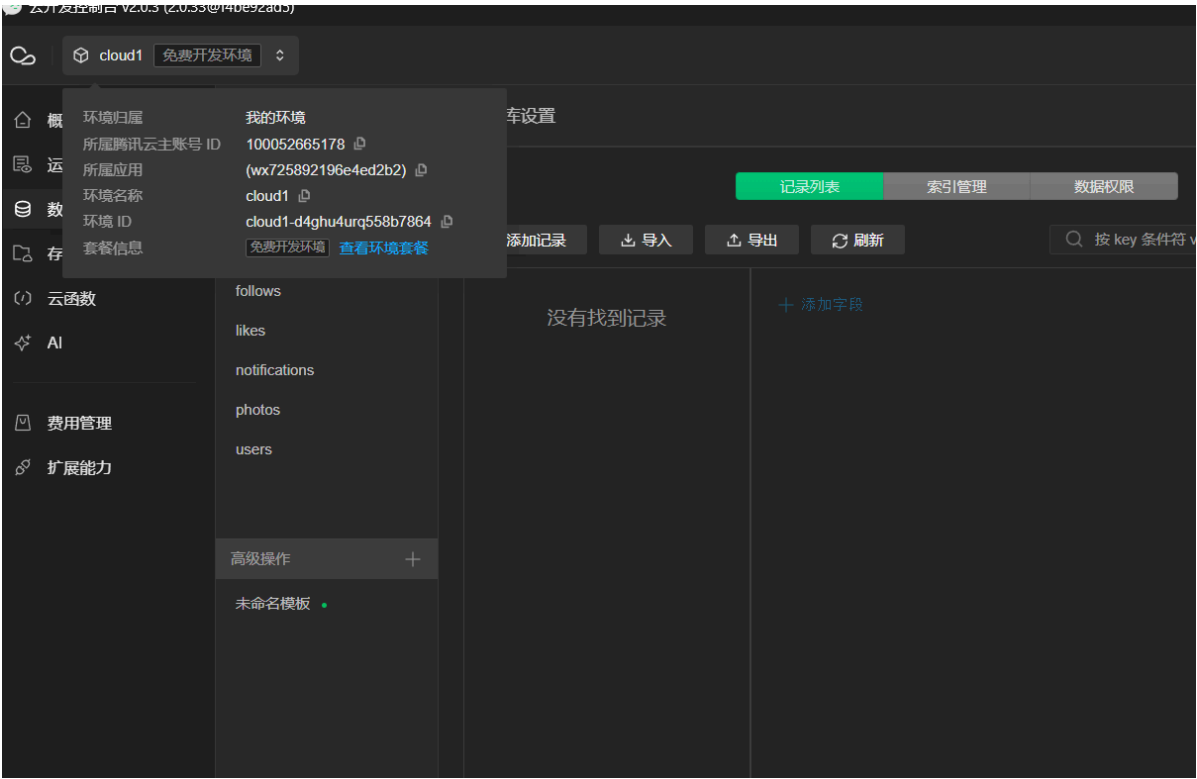
1. 实验目标与开发环境

本实验使用微信小程序原生框架和微信云开发能力，完成一个图片分享社区 `cCloudPhoto`。项目综合使用云数据库、云存储和云函数，实现图片上传、图片流展示、作者主页、图片详情、图片保存、分享、照片删除和会话退出等功能。

在教材基础功能的基础上，项目进一步完善了分页加载、下拉刷新、搜索、关注、点赞、收藏、评论、上传进度、空状态、错误重试、照片删除、退出登录和孤立云文件清理等交互。主要练习内容包括：

- 使用 `wx.cloud.init()` 初始化云开发环境；
- 使用云数据库 `photos` 集合保存图片元信息；
- 使用云存储上传、下载和清理用户图片；
- 使用 `getOpenid` 云函数获取当前用户的 `OPENID`；
- 使用原生小程序页面和自定义 `TabBar` 组织社区界面；
- 处理用户资料授权、系统选图面板和相册权限；
- 通过本地状态模块实现关注、点赞、收藏、评论和未读消息体验；
- 实现作者照片删除、云端文件清理和本地会话退出。

项目开发环境为微信开发者工具、微信小程序原生框架和微信云开发，项目 AppID 为 `wx725892196e4ed2b2`。



2. 项目结构与页面功能

小程序以社区首页为主入口，并通过自定义 TabBar 提供“社区”和“我的”两个主要入口。首页展示所有用户的图片流，个人主页展示指定作者的作品，详情页提供图片操作和互动，上传页负责发布图片。

页面	路径	主要功能
社区首页	<code>pages/index/index</code>	图片流、社区/关注切换、搜索、分页、下拉刷新和上传入口
我的页面	<code>pages/mine/mine</code>	用户资料、作品管理、删除照片、退出登录和个人入口
消息页面	<code>pages/messages/messages</code>	评论、互动和未读消息展示
个人主页	<code>pages/homepage/homepage</code>	作者资料、作者作品列表和关注操作
图片详情	<code>pages/detail/detail</code>	图片展示、下载、预览、分享、点赞、收藏、评论和作者删除
上传图片	<code>pages/add/add</code>	用户授权、选图上传、文案填写和历史图片墙

主要目录如下：

1	code6/	
2	├─ miniprogram/	
3	└─ app.js / app.json / app.wxss	
4	└─ custom-tab-bar/	# 社区与我的自定义底部导航
5	└─ pages/	
6	└─ index/	# 社区首页

7				mine/	# 我的页面
8				messages/	# 消息页面
9				homepage/	# 作者个人主页
10				detail/	# 图片详情页
11				add/	# 上传图片页
12				utils/	
13				photo.js	# 日期、图片规范化和错误提示
14				social.js	# 关注、点赞、收藏、评论、未读状态和会话清理
15				cloudfunctions/	
16				getOpenid/	# 获取当前用户 OPENID

这种结构将云数据请求、图片处理和社区互动状态分开，页面主要负责展示与交互，公共工具负责可复用逻辑。



3. 全局配置与视觉主题

`app.json` 注册社区相关页面，配置自定义 TabBar，并将导航栏设置为浅色背景和黑色文字：

```
1  {
2    "pages": [
3      "pages/index/index",
4      "pages/mine/mine",
5      "pages/messages/messages",
6      "pages/homepage/homepage",
7      "pages/detail/detail",
8      "pages/add/add"
9    ],
10   "tabBar": {
11     "custom": true,
12     "color": "#718078",
13     "selectedColor": "#246B4B",
14     "backgroundColor": "#FFFD8"
15   },
16   "window": {
17     "navigationBarBackgroundColor": "#F7F4EE",
18     "navigationBarTitleText": "图片分享社区",
19     "navigationBarTextStyle": "black"
20   }
21 }
```

`app.wxss` 统一定义页面、卡片、头像、按钮、状态提示和浮动按钮样式。整体采用米白色背景、深绿色主色、金色辅助色和珊瑚红提示色，卡片使用圆角、细边框和轻阴影，形成偏生活方式社区的视觉风格。

页面底部预留安全区域，内容容器增加底部间距，避免自定义 TabBar 和浮动上传按钮遮挡最后一张图片。

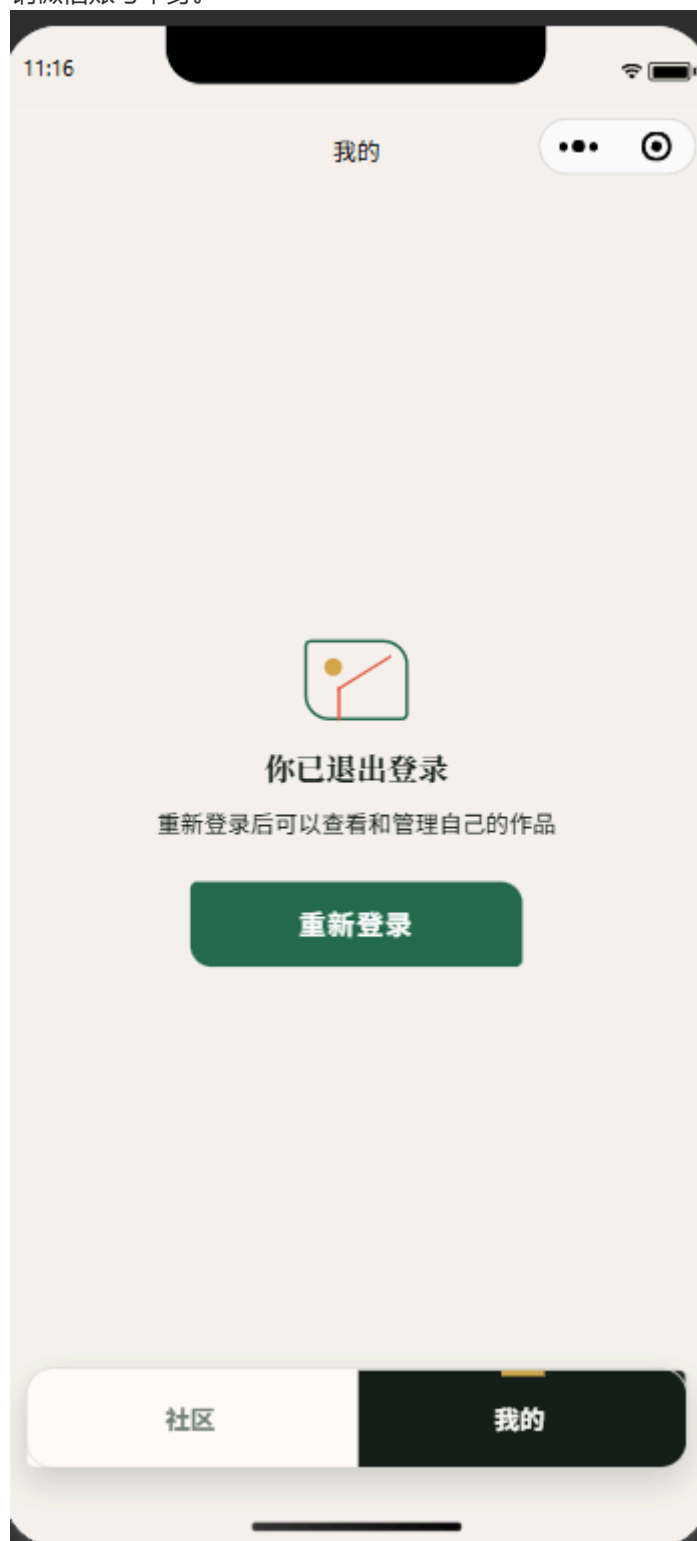
4. 云开发初始化与用户身份

`app.js` 在小程序启动时初始化云能力，并在全局状态中保存用户资料、OPENID 以及正在进行的 OPENID 请求：

```
1  App({
2    onLaunch: function () {
3      this.globalData = {
4        userInfo: null,
5        openid: null,
6        openidPromise: null,
7        loggedOut: false
8      }
9
10     wx.cloud.init({ traceUser: true })
11   }
12 })
```

`requestUserProfile()` 使用 `wx.getUserProfile()` 获取展示头像和昵称所需的用户资料；`ensureOpenId()` 调用 `getOpenid` 云函数，并缓存 Promise，避免用户连续操作时重复请求身份信息。上传前会串行完成用户授权和 OPENID 获取，再打开系统选图面板，避免多个原生弹窗同时出现。

项目还提供 `clearSession()`，退出时清空全局用户资料、OPENID 和 OPENID 请求缓存，并设置会话退出标记。由于微信小程序 CloudBase 使用微信原生身份，退出登录表示退出本应用会话，不会注销微信账号本身。





5. 云数据库与云存储设计

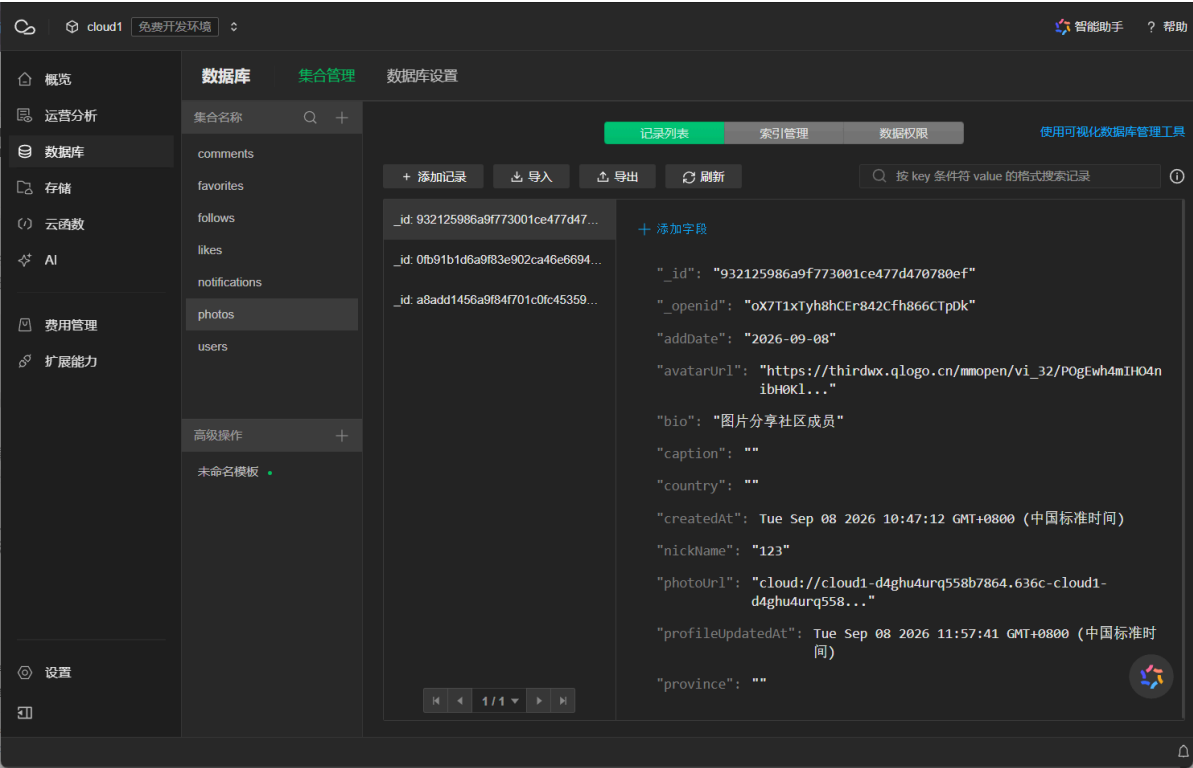
项目使用名为 `photos` 的云数据库集合。云开发会自动注入 `_id` 和 `_openid`，前端写入的主要字段如下：

字段	作用
<code>photoUrl</code>	云存储图片文件 ID
<code>avatarUrl</code>	图片作者头像
<code>country</code> / <code>province</code>	图片作者所在地区
<code>nickName</code>	图片作者昵称

字段	作用
<code>addDate</code>	兼容教材要求的日期字符串
<code>createdAt</code>	云端时间，用于可靠排序
<code>caption</code>	用户填写的可选图片文案

集合权限设置为“所有用户可读，仅创建者及管理员可写”。用户上传图片时，文件以 `photos/OPENID/时间戳-随机数.扩展名` 的形式保存到云存储，再将返回的 `fileID` 写入数据库。照片作者可以删除自己的照片，删除流程先清理云存储文件，再删除对应的 `photos` 数据记录；数据库权限仍负责从云端校验创建者身份。

数据写入失败时，程序会调用 `wx.cloud.deleteFile()` 删除刚刚上传的文件，避免云存储中留下没有数据库记录的孤立文件。集合中的 `_openid` 由云开发自动维护，不由前端手动写入。



6. 社区首页与图片流

首页在 `onShow()` 中加载图片列表，因此从上传页返回时可以及时看到最新发布内容。图片按照 `createdAt` 倒序排列，单次加载 8 条数据，使用 `skip()` 和 `limit()` 实现分页，并通过 `onReachBottom()` 加载更多。

首页还实现了下拉刷新、空状态、加载状态和错误重试。搜索框会根据作者昵称、所在地和图片文案进行本地过滤；“社区”和“关注”两个 feed 标签分别显示全部图片或已关注作者的图片。

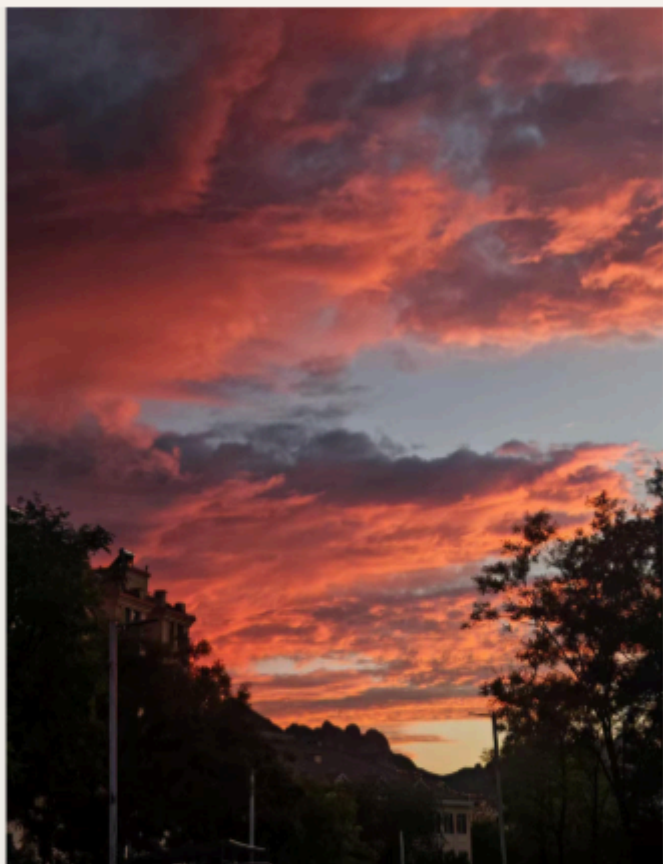
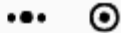
```
1  photos
2    .orderBy('createdAt', 'desc')
3    .skip(currentList.length)
4    .limit(that.data.pageSize)
5    .get()
6    .then(function (res) {
7      var nextList = currentList.concat(res.data)
8      that.setData({
9        photoList: nextList.map(photoUtils.normalizePhoto),
10        hasMore: res.data.length === that.data.pageSize
11      })
12    })
```

每张图片卡片包含作者头像、昵称、所在地、序号、图片、文案、发布日期以及点赞和收藏操作。点击头像进入个人主页，点击图片进入详情页。

11:19



图片分享社区



晚霞



25



0



收藏

发布于 2026-09-09



微信用户

来自图片分享社区



上传图片

社区

我的

11:19



图片分享社区



图片分享社区

今日影像



晚霞

清除

关注

社区



微信用户

NO.01

来自图片分享社区



上传图片

社区

我的

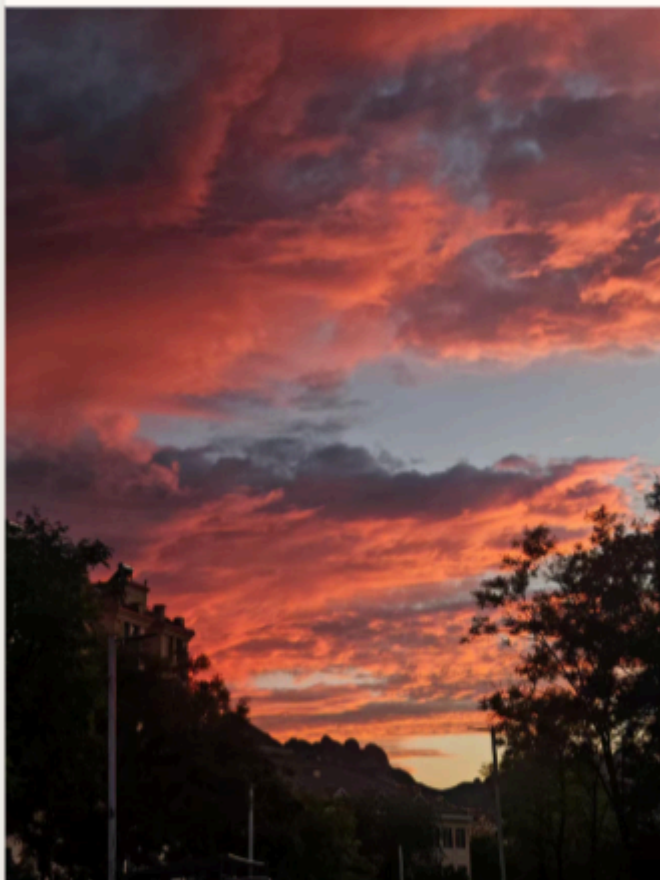
11:19



图片分享社区



不日国月力季秋金



晚霞



15



0



收藏

发布于 2026-09-09

已经看到社区

上传图片

社区

我的

11:20



图片分享社区



图片分享社区



今日影像

🔍 搜索作者、地点或作品

关注

社区



关注的人还没有新作品

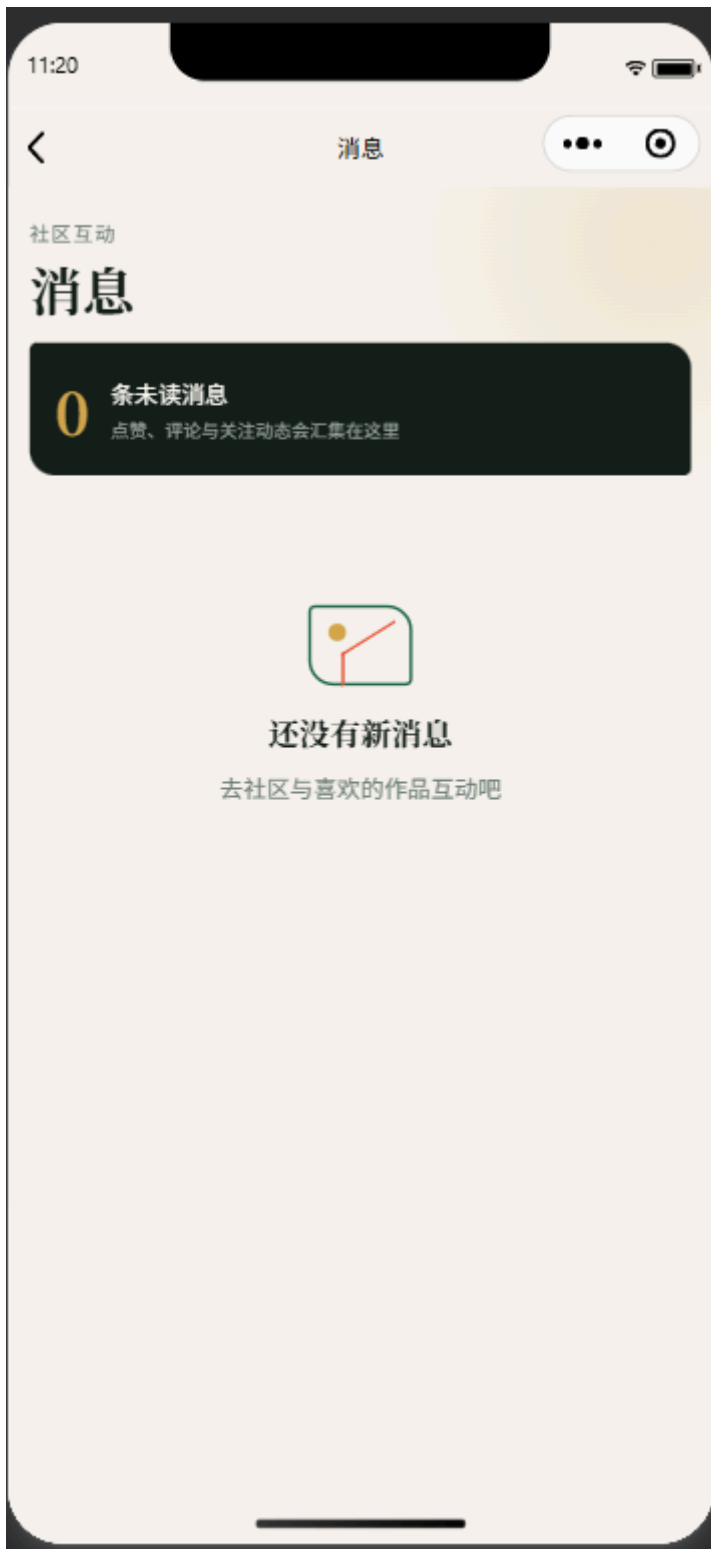
去社区发现喜欢的作者吧

已经看到社区的第一张照片

⊕ 上传图片

社区

我的



7. 上传图片与历史图片墙

上传页使用 `wx.chooseImage()` 从相册或相机选择一张压缩图片，并通过 `uploadTask.onProgressupdate()` 展示实时上传进度。用户可以在上传前填写可选图片文案，上传成功后同时创建数据库记录并刷新个人历史图片墙。

```
1 var cloudPath = 'photos/' + openid + '/' +  
2   Date.now() + '-' + Math.floor(Math.random() * 10000) + '.' + extension  
3  
4 wx.cloud.uploadFile({  
5   cloudPath: cloudPath,  
6   filePath: filePath,  
7   success: resolve,  
8   fail: reject  
9 })
```

上传按钮具有防重复提交机制，上传过程中会显示进度并暂时禁止再次操作。完成后提示“发布成功”，清空文案输入框并刷新当前用户的历史图片。历史图片按创建时间倒序排列，点击图片可调用 `wx.previewImage()` 进行预览。

8. 个人主页与自定义 TabBar

个人主页从首页卡片头像携带 `_openid` 跳转进入，通过 `where({ _openid: openid })` 查询指定用户的作品，并在顶部显示作者头像、昵称和关注状态。页面同样支持分页加载、错误重试和关注/取消关注。

自定义 TabBar 将“社区”和“我的”固定在底部，并在首页和我的页面的 `onShow()` 中同步当前选中项。公共页面容器预留底部空间，使列表滚动到末尾时仍能完整显示。

```
1 <navigator url="../../homepage/homepage?id={{item._openid}}">  
2   <image class="avatar" src="{{item.avatarUrl}}" mode="aspectFill"></image>  
3 </navigator>
```



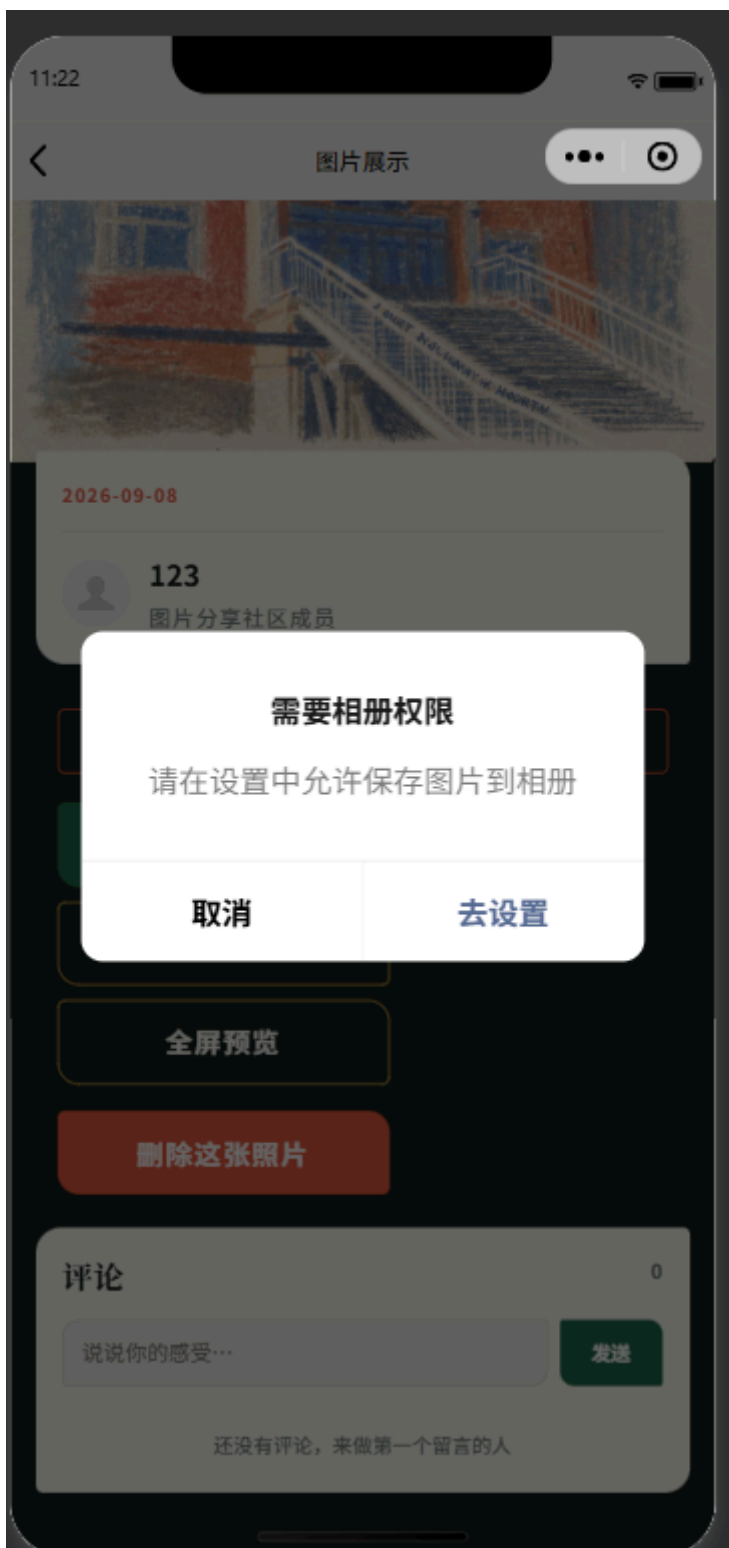
9. 图片详情、下载、预览与分享

图片详情页通过 `_id` 查询单条图片记录，正常加载时显示大图、作者信息、文案和互动区域。用户可以执行以下操作：

- 使用 `wx.cloud.downloadFile()` 下载云存储文件，再保存到手机相册；
- 使用 `wx.previewImage()` 全屏预览图片；
- 通过 `onShareAppMessage()` 分享给好友；
- 通过 `onShareTimeline()` 分享到朋友圈；
- 点赞、收藏并查看或发表评论。

分享内容会优先使用图片文案作为标题，并携带当前图片的详情页路径。下载失败时会区分相册权限不足和模拟器没有系统相册两种情况：真机引导用户打开设置，模拟器则提示保存失败属于正常现象。





10. 社区互动状态

`utils/social.js` 对关注、点赞、收藏、评论和消息状态进行统一管理。首页和详情页通过 `decoratePhoto()` 将当前用户的互动状态合并到图片数据中，交互后只刷新当前展示数据，不需要重新请求整个图片列表。

关注状态用于“关注”图片流筛选；点赞和收藏状态用于卡片与详情页的图标和数量反馈；评论数据用于详情页评论区；未读消息数量同步显示在首页和消息入口。

这些互动状态属于本地体验数据，不改变云数据库中图片的核心记录，避免在实验基础功能之外引入不必要的云端写权限。

退出登录时，`social.clearSession()` 会移除本地保存的关注、点赞、收藏、评论、通知和资料状态，防止下一次使用时继续显示上一位用户的本地互动数据。退出后“我的”页面显示重新登录入口；重新完成用户资料授权和 OPENID 获取后，页面恢复作品管理功能。

11. 照片删除与会话退出

“我的”页面的每张作品卡片都提供删除按钮，图片详情页仅在当前 OPENID 与照片 `_openid` 一致时显示删除按钮。删除前弹出二次确认，确认后将照片从云存储和数据库中移除，并刷新作品列表；删除过程中按钮进入 loading 状态，避免重复提交。

```
1 var removeFile = photo.photoUrl
2   ? wx.cloud.deleteFile({ fileList: [photo.photoUrl] })
3   : Promise.resolve()
4
5 Promise.resolve(removeFile).then(function () {
6   return photos.doc(photo._id).remove()
7 })
```

详情页在加载公开照片时不会因为身份判断失败而阻止照片展示，只会隐藏删除入口。这样公开浏览和作者管理彼此独立，同时依赖 `photos` 集合权限保护实际删除操作。

退出登录按钮位于“我的”页面底部。用户确认后，页面清空当前作品和资料，并显示“你已退出登录”；点击“重新登录”后重新请求用户资料和 OPENID。云端照片不会因退出登录而删除。





12. 错误处理与加载状态

项目为列表页、上传页、个人主页和详情页分别设置了加载标记、空状态和错误信息。云数据库请求失败时，页面不会停留在空白状态，而是显示用户可理解的提示，并提供下拉刷新或“重新加载”操作。

`photoutils.friendlyError()` 将底层错误转换成更适合用户阅读的中文提示；上传流程会处理用户取消选图、授权失败、网络失败和数据库写入失败等情况。加载和上传结束后统一关闭 loading 状态，避免页面一直显示遮罩。

13. 云函数 `getOpenid`

云函数位于 `cloudfunctions/getOpenid/index.js`，使用动态云环境并通过 `cloud.getWXContext()` 获取当前调用者的 OPENID：

```
1  const cloud = require('wx-server-sdk')
2
3  cloud.init({
4    env: cloud.DYNAMIC_CURRENT_ENV
5  })
6
7  exports.main = async () => {
8    const wxContext = cloud.getWXContext()
9    return { openid: wxContext.OPENID }
10 }
```

部署时需要在微信开发者工具中右键 `getOpenid` 文件夹，选择“上传并部署：云端安装依赖”。前端调用成功后将 OPENID 缓存在 `app.globalData` 中，后续查询个人历史图片和个人主页时使用该身份标识。



二、问题总结与体会

1. 实验中遇到的问题及解决方法

(1) **用户授权弹窗和系统选图面板同时弹出。** 两个原生面板同时调用时可能互相抢占，导致其中一个无法正常显示。解决方法是将授权、获取 OPENID 和选图操作串成 Promise 链，前一步完成后再打开下一步。

(2) **图片上传成功但数据库写入失败会留下无效文件。** 云存储上传和数据库写入是两个独立操作。解决方法是在数据库写入失败的 catch 分支中调用 `wx.cloud.deleteFile()` 清理刚上传的文件。

(3) **仅使用日期字符串排序时，同一天的图片顺序不稳定。** 教材字段 `addDate` 只有年月日，无法区分同一天内的多次上传。解决方法是增加 `createdAt: db.serverDate()`，优先按云端时间排序，并保留 `addDate` 兼容教材要求。

(4) **模拟器无法保存图片到相册。** 模拟器通常没有真实系统相册。解决方法是区分权限错误和模拟器保存失败，真机提示用户授权，模拟器显示“无相册时属正常现象”的说明。

(5) **页面内容可能被自定义 TabBar 或浮动上传按钮遮挡。** 解决方法是在公共 `.container` 中增加底部 padding，并将浮动按钮放置在安全区域上方，保证列表末尾和操作按钮都可见。

(6) **删除照片需要同时处理云存储和数据库。** 如果只删除数据库记录，云存储中会留下无法访问的文件；如果只删除云文件，社区中仍可能出现失效图片。解决方法是删除前进行作者身份判断和二次确认，按顺序删除云文件与数据库记录，并在操作过程中锁定按钮防止重复提交。

2. 实验收获与体会

通过本次实验，我完成了一个基于微信云开发的图片分享社区，理解了小程序前端页面、云数据库、云存储和云函数之间的协作关系。图片文件适合保存在云存储，图片作者和日期等元信息适合保存在云数据库，而用户身份则通过云函数从微信运行环境中安全获取。